

Sportsbook Customer Economics

Robert Carrington · September 2026

Abstract

This report estimates what a sportsbook's first-time depositors (FTDs) cost to acquire, what they contribute in their first year, and how early their value can be predicted. The player-level data is a simulation of 40,000 FTDs whose seasonality, hold, and hold volatility are calibrated to DraftKings' public monthly filings in New York. Year-one contribution averages \$585 per FTD against a blended CAC of \$323, a $1.81\times$ return, but value is highly concentrated: the top 1% of FTDs produce 46% of it and 66% are net negative at day 180. Channel, welcome offer, and state tax each move payback materially. An XGBoost model on the first 14 days of activity ranks players' 180-day value with a Spearman correlation of 0.57, against 0.37 for sorting by early handle alone, and a two-part version improves that to 0.58 while fixing most of the model's under-prediction for top players. A retention-decay model, backtested within 5% of actual 12-month value from six months of data, projects 36-month returns of $3.1\times$ to $9.4\times$ CAC across channels. The report also sizes the experiments needed to act on its recommendations. A separate section tests TabPFN, a pretrained tabular foundation model. Given 3,000 training players and no tuning, it ranks 180-day value at 0.59 Spearman, better than XGBoost trained on all 33,246, and comes within 0.008 AUC of it on retention.

I built this for the Analyst I, Customer Economics role at DraftKings. None of it uses DraftKings internal data, and where a result follows directly from an assumption rather than from the analysis, I say so.

Headline	Value
Simulated FTDs	40,000
Blended CAC	\$323
12-month contribution per FTD	\$585
12-month LTV / CAC	$1.81\times$
Share of FTDs net negative at day 180	66%
Share of year-one contribution from the top 1%	46%

Contents

1. Objective
2. Data collection
3. Data cleaning and transformation

4. Exploratory data analysis
5. Feature engineering
6. Modeling choices
7. Findings
8. Test design for the recommended changes
9. Conclusion
10. TabPFN: process and comparison
11. Reproducibility

1. Objective

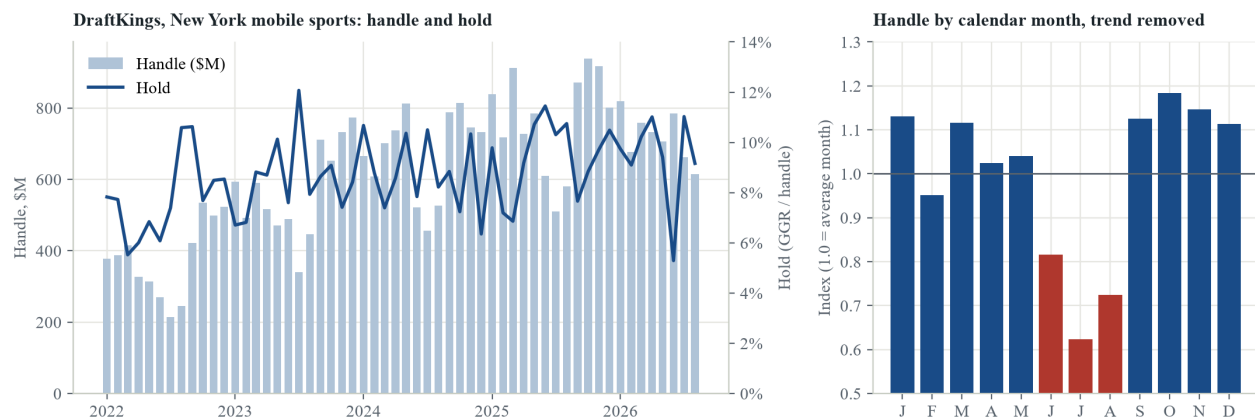
A customer economics team decides how much to pay for a new customer and where to spend to keep them. This analysis answers four questions:

1. What is a first-time depositor worth over 12 months, after promos, tax, and variable cost?
2. How do acquisition channel, welcome offer, and state change that value and the payback period?
3. How do retention and value evolve by cohort?
4. How well can a player's value be predicted from their first 14 days, and which model should do it?

2. Data collection

2.1 Real data: DraftKings' New York filings

The New York State Gaming Commission publishes each mobile sports operator's monthly handle (total amount wagered) and gross gaming revenue (GGR, the amount the book keeps). I downloaded DraftKings' report, a PDF covering January 2022 through August 2026, 56 months in all. Over the last 12 months DraftKings took \$9.28B in New York handle at a 9.3% hold, and New York taxes GGR at 51%.



2.2 Simulated player-level data

No public dataset has sportsbook customers with acquisition channel, CAC, and promo cost, which is the core of customer economics. The one academic dataset of real bettors (the Transparency Project's bwin data) is licensed for non-commercial research only. So I simulated the player level, and used the real filings to calibrate it.

`simulate.py` generates 40,000 FTDs who signed up between September 2024 and August 2025 and records every day each one bet through June 2026, 1,563,923 player-days in total. Each FTD has:

- an acquisition channel (referral, TV and brand, search, affiliate, paid social), each with its own CAC
- a welcome offer (no-sweat first bet, deposit match, or bet \$5 get \$200), each with its own expected cost
- a state (nine, each with its tax rate)
- a hidden player type (bonus hunter, casual, regular, high value) that sets churn, betting frequency, stake size, odds preference, and how much of the welcome offer the player extracts

Channels and offers shift the mix of player types. That is the main assumption driving channel and offer results. The analysis never sees player type. It works only from behavior, as it would on real data. Type is used once, at the end, to check whether the models find the right players.

What is calibrated to real data, and how close the simulation lands:

Measure	DraftKings NY (real)	Simulation
Hold, Sep 2024 onward	9.1%	9.5%
Monthly hold, standard deviation	1.64 pts	1.55 pts
Seasonal pattern of activity	Index by calendar month (2.1)	Same index, used as input
New York tax rate	51% of GGR	51%

Hold swings month to month because every customer bets on the same games, so outcomes are correlated. The simulation adds one shared shock per month, sized to the real standard deviation. Channel CACs, offer costs, player-type behavior, and the non-New York tax rates are my assumptions, all listed in `assumptions.toml`.

3. Data cleaning and transformation

3.1 Regulator filings

- Extracted the monthly rows from each page of the PDF with `pdfplumber` and a regular expression, then dropped the pre-launch months reported as \$0.
- De-duplicated months that appear on more than one fiscal-year page.
- Computed hold as GGR divided by handle, and checked the state's share of GGR against the statutory 51% (it matches to three decimals every month).
- Removed market growth before measuring seasonality: each month's handle was divided by a centered 12-month moving average, and the ratios were averaged by calendar month and scaled to a mean of 1.

3.2 Player data

All transformation is in SQL (DuckDB), in `sql/01` through `sql/15`. The central table is a **player-month panel**: one row per FTD per 30-day "life month" since first deposit, 686,051 rows in all. Three decisions shape it:

- **Zero months are kept.** A month with no bets is a row with zeros, not a missing row. Averages are per FTD, not per active player, so retention and value are not overstated.
- **Right censoring is handled by truncation.** Each player is cut at their last fully observed month. Any 12-month figure uses only players with 12 full months, which is 36,457 of the 40,000 FTDs. Younger cohorts never pull a curve down.
- **Life months, not calendar months.** Month 1 is each player's first 30 days, so cohorts that signed up at different times are compared at the same age.

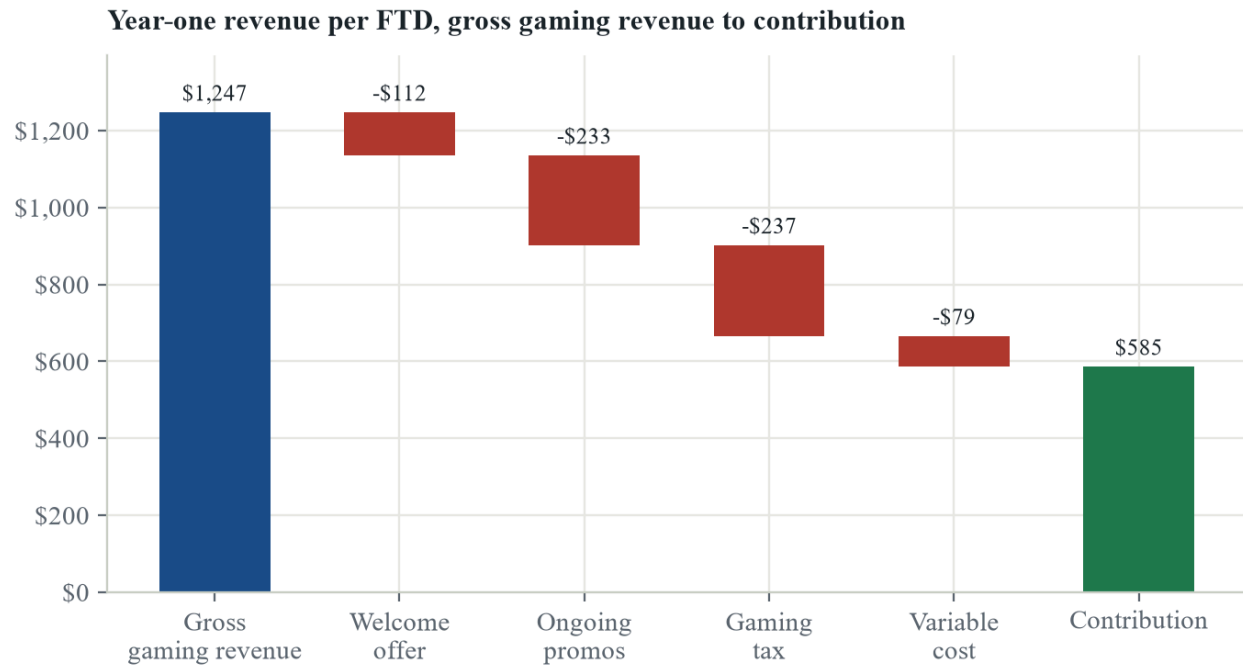
Contribution is the value measure throughout:

```
contribution = GGR - welcome promo - ongoing promos - tax x (GGR - promos) -  
0.6% x handle
```

It is a contribution margin, not profit: it excludes fixed costs and overhead. Tax is applied to GGR net of promos as a simplification (section 9.3).

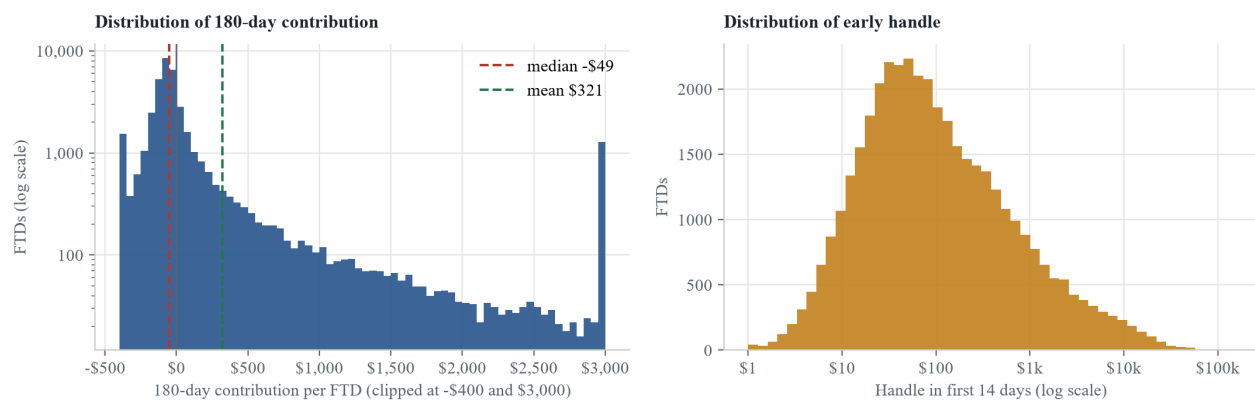
4. Exploratory data analysis

4.1 Where revenue goes



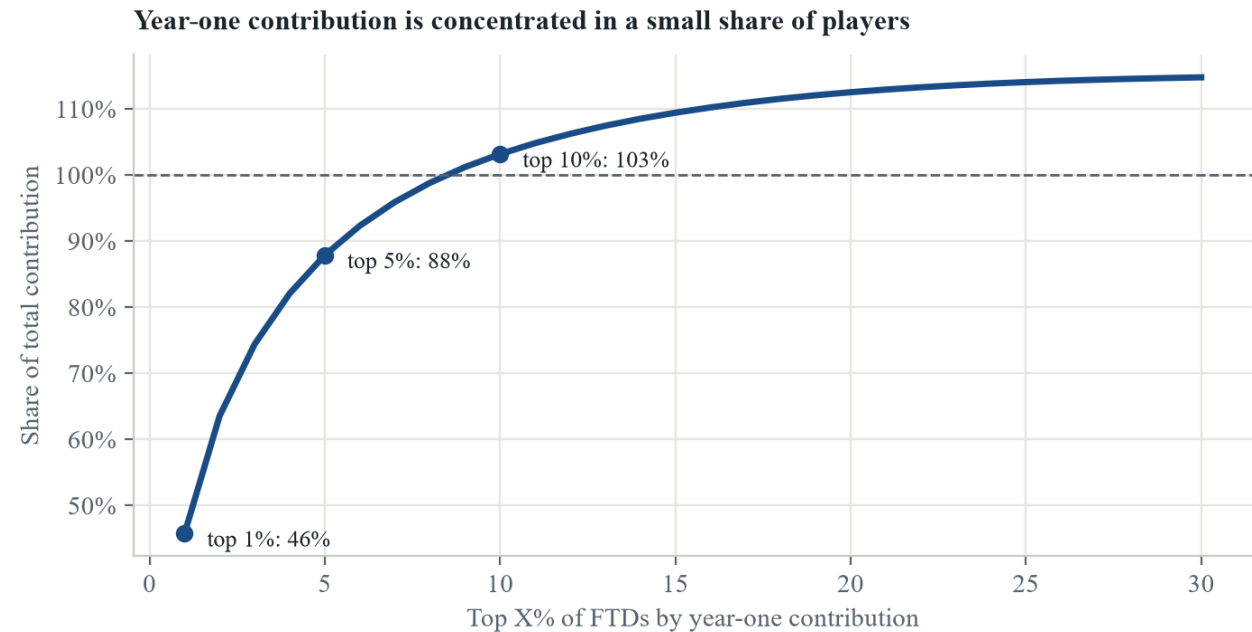
Of \$1,247 in year-one GGR per FTD, promos take \$345 (28%) and tax \$237. Contribution is \$585, 47% of GGR.

4.2 The distribution of player value



Mean	Median	10th pct	25th pct	75th pct	90th pct	99th pct	Skewness
\$321	-\$49	-\$188	-\$110	\$72	\$675	\$8,311	18.3

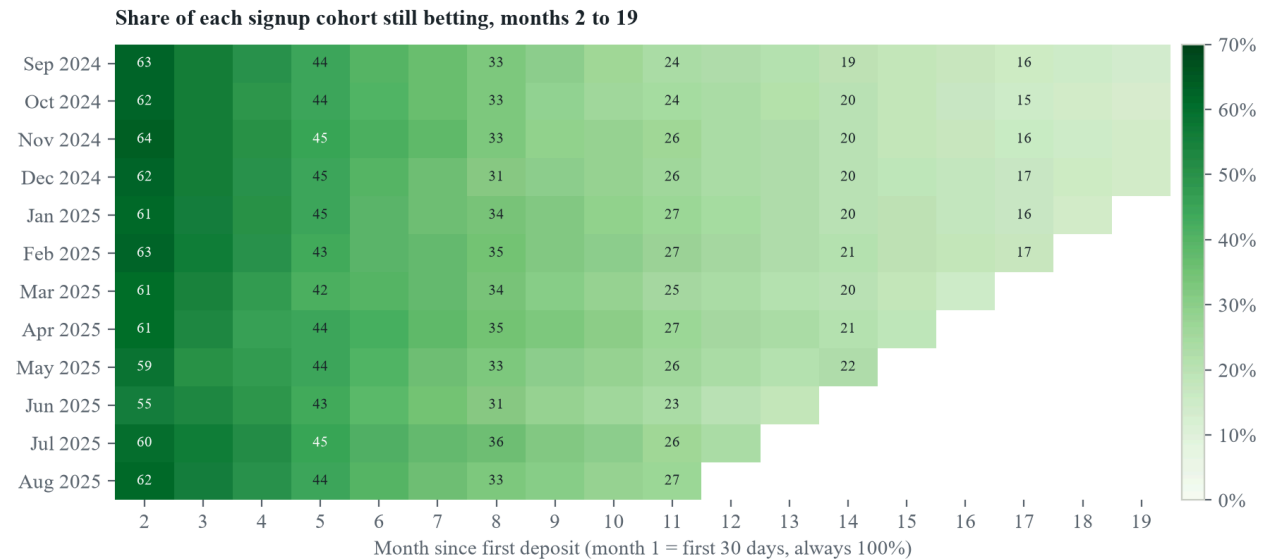
180-day contribution is extremely right-skewed. The mean is \$321 while the median is -\$49, and 66% of FTDs are below zero, mostly because the welcome offer costs more than the book ever wins back from them. This shapes every later choice: averages are fragile, so channel results get bootstrap intervals, and model evaluation leans on rank-based metrics rather than squared error.



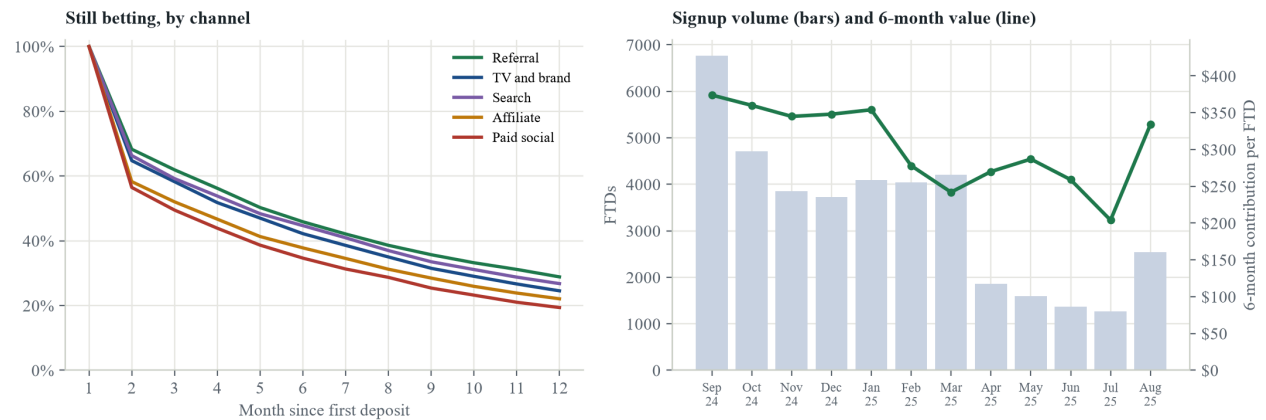
Players	Share of year-one contribution	Average contribution
Top 1%	46%	\$26,755
Top 5%	88%	\$10,273
Top 10%	103%	\$6,031
Top 20%	112%	\$3,289

The top 9% of players account for all year-one contribution. The share passes 100% because the rest are net negative in aggregate.

4.3 Retention



On average 39% of FTDs place no bet in their second month and 23% are still betting in month 12. Seasonality shows up in the second month: 64% of November 2024 signups bet again, against 55% of June 2025 signups, whose second month is July.



FTDs who sign up between September and January are worth \$359 over six months, against \$259 for February through July signups, whose early months overlap the summer lull.

4.4 Early behavior against later value

Handle, days 0 to 13	Share of FTDs	Avg active days	Active month 3	Mean 180-day contribution	Share profitable	Share of total 180- day
under \$50	39.7%	1.7	36%	-\$74	8%	-9%
\$50 to \$250	30.9%	2.6	54%	-\$30	33%	-3%
\$250 to \$1k	16.7%	4.1	78%	\$164	60%	8%
\$1k to \$5k	9.0%	5.8	88%	\$1,108	78%	31%
\$5k and up	3.7%	7.5	92%	\$6,359	87%	73%

Early handle separates players sharply. The 3.7% of FTDs who stake \$5,000 or more in their first two weeks produce 73% of 180-day contribution. This is the baseline any model has to beat.

5. Feature engineering

The prediction point is **day 14**: a model scores each FTD using only what is known by the end of their 14th day (days 0 to 13). Features are built in `sql/08_early_features.sql`.

Targets

- `active_m3`: whether the player bets at all in life month 3 (days 60 to 89). Binary, 55% positive.
- `contribution_180`: total contribution over days 0 to 179. It includes the first 14 days by design, because the business question is the player's total value. The model's real work is the remaining 166 days.

Features, with their rank correlation to each target

Feature	Spearman with 180-day contribution	Spearman with active in month 3
Active days, days 0 to 13	0.30	0.35
Active days, days 7 to 13	0.28	0.33
Bets placed	0.37	0.42
Handle (total staked)	0.40	0.40
GGR (book's win)	0.42	0.10
Largest single-day handle	0.40	0.37
Average stake per bet	0.34	0.27
Days since last bet, at day 13	-0.24	-0.31
Welcome offer cost	-0.24	-0.02

- Categorical context: acquisition channel, welcome offer, state, and signup month (for seasonality).
- Week 2 activity and days idle are included because the *trend* of engagement matters: a player who bet heavily on day 1 and then stopped differs from one still betting on day 13.
- Largest single-day handle is included alongside total handle because a few large days signal a high-stakes player differently from many small ones.
- **Leakage check:** no feature uses anything after day 13, and nothing downstream of the outcome (such as churn date or lifetime) is available to the model.

Transformations depend on the model. For logistic regression, numeric features get a signed log transform, $\text{sign}(x) * \log(1 + |x|)$, to tame the heavy tails (GGR can be negative), then standardization, and categoricals are one-hot encoded. XGBoost takes raw values and native categorical splits, since trees are invariant to monotone transforms.

6. Modeling choices

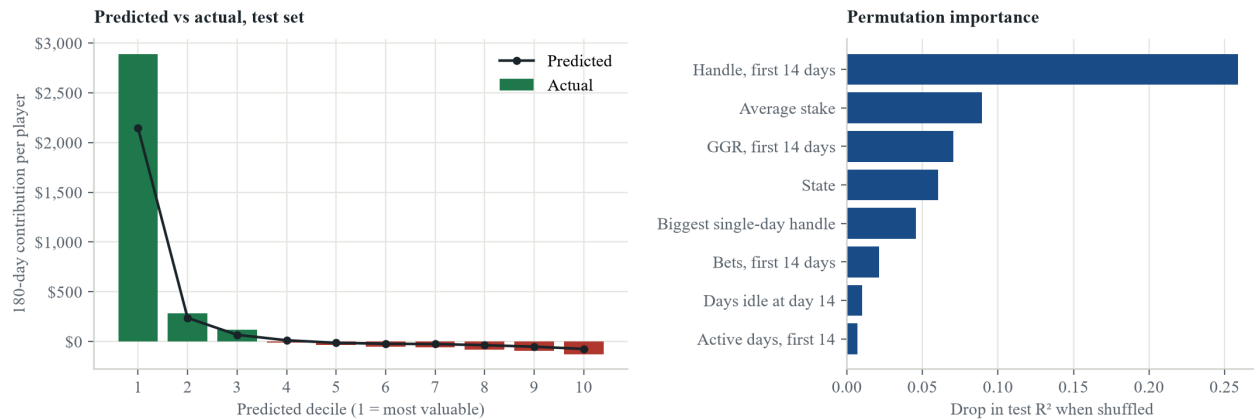
- **Out-of-time split.** Train on Sep 2024 to Apr 2025 signups (33,246 players), test on May 2025 to Aug 2025 (6,754). A random split would leak seasonal information across the boundary, and in practice the model is fit on past cohorts and used on new ones.
- **Baselines first.** Logistic regression for retention, and two naive rules for value: sort by 14-day handle, and predict the training mean.
- **XGBoost** as the main model: histogram trees with native categorical splits, which handle missing values, categoricals, and interactions without manual work. It is the standard strong baseline for tabular data. Settings: learning rate 0.03, max depth 6, minimum child weight 5, row and column subsampling of 0.8, and early stopping after 100 rounds without improvement on a held-out 10% of

the training rows, so the number of trees is chosen by validation. Squared-error objective for value, log loss for retention.

- **Metrics chosen for the skew.** AUC for retention (ranking quality, insensitive to the threshold). For value, Spearman rank correlation and top-decile lift, because the business action is ranking players and a few whales dominate any squared-error metric. Mean absolute error is reported for completeness.
- **Permutation importance** on the test set, which measures how much accuracy drops when a feature is shuffled and is less biased toward high-cardinality features than split-based importance.
- **Bootstrap intervals** for channel LTV/CAC: 2,000 resamples of players within each channel, 95% percentile intervals. Given the skew in 4.2, point estimates alone would overstate precision.
- **A two-part value model** as a second specification, because squared-error loss shrinks a heavy right tail. One XGBoost classifier estimates the chance a player ends up profitable. A second XGBoost model predicts $\log(1 + \text{value})$ for profitable players, converted back to dollars with Duan's smearing correction estimated on held-out rows. A third predicts the size of the loss for unprofitable players. Expected value is $p \times E[\text{value} \mid \text{profitable}] + (1 - p) \times E[\text{value} \mid \text{not profitable}]$.
- **Calibration checks.** Brier score and reliability curves (predicted probability against the actual share, by decile) for the retention models, and predicted against actual value by decile for the value models. Ranking metrics say nothing about whether a predicted \$500 is really \$500, and bids are set in dollars.
- **Lifetime value beyond the observed window** with a shifted-beta-geometric (sBG) retention curve per channel: the share of FTDs still betting in month t is $S(t) = B(a, b + t) / B(a, b)$, which allows churn to differ across players and gives the long, slow-decaying tail a single churn rate cannot. Projected contribution is $S(t)$ times the value of an active player (the average of the last three observed months). The method is backtested twice on held-out months before it is used, with bootstrap intervals on the projections.
- **Test design** for the recommended changes: sample size per arm at 5% significance and 80% power, winsorizing the outcome at the 99th percentile to tame the tail, and CUPED or regression adjustment on pre-treatment covariates to reduce variance.

7. Findings

7.1 Model performance



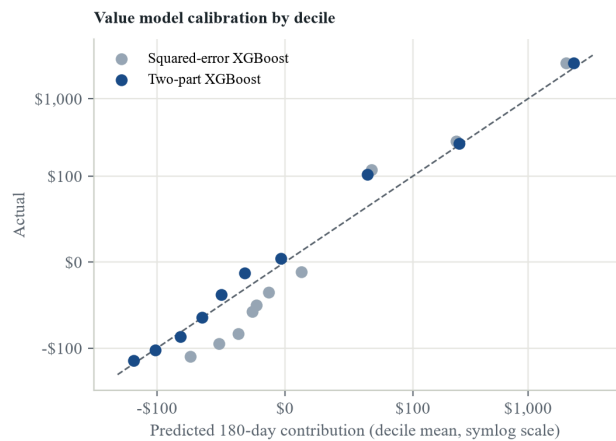
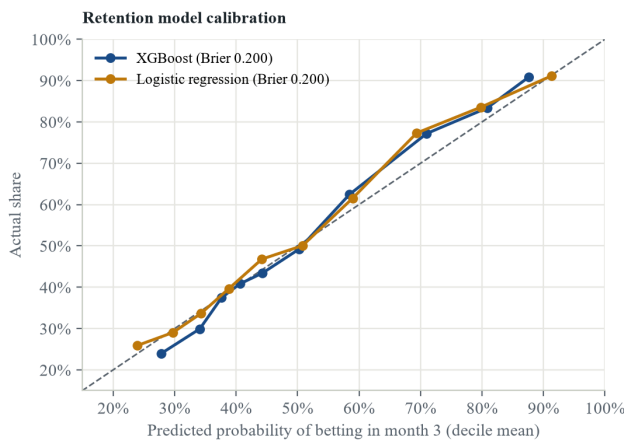
Question	Metric	XGBoost	Baseline
Bets in month 3?	AUC	0.753	0.755 (logistic regression)
Flag the at-risk decile	Share who lapse	76%	46% (all players)
Rank by 180-day value	Spearman	0.572	0.367 (sort by 14-day handle)
Find the top decile	Lift over average	10.20×	10.07× (sort by 14-day handle)
Predict dollar value	Mean absolute error	\$379	\$690 (predict the mean)
Find hidden high-value players	Share of top decile	72%	10% (base rate)

- On retention, logistic regression matches XGBoost (0.755 vs 0.753 AUC). The simpler model is the one to ship there.
- On value, XGBoost ranks the whole base better than early handle alone (0.57 vs 0.37), but ties it on the top decile. Use the model for bids and retention spend across everyone, and a handle threshold for routing likely high-value players to VIP.
- The squared-error value model is biased low at the top: it predicts \$2,147 for its top decile, which actually averages \$2,890. The two-part model below addresses this.
- 72% of the model's top decile are truly high-value players, against a 10% base rate, which confirms it is finding the right people rather than fitting noise.

Two-part value model. Same features, same split, three XGBoost models combined as described in section 6.

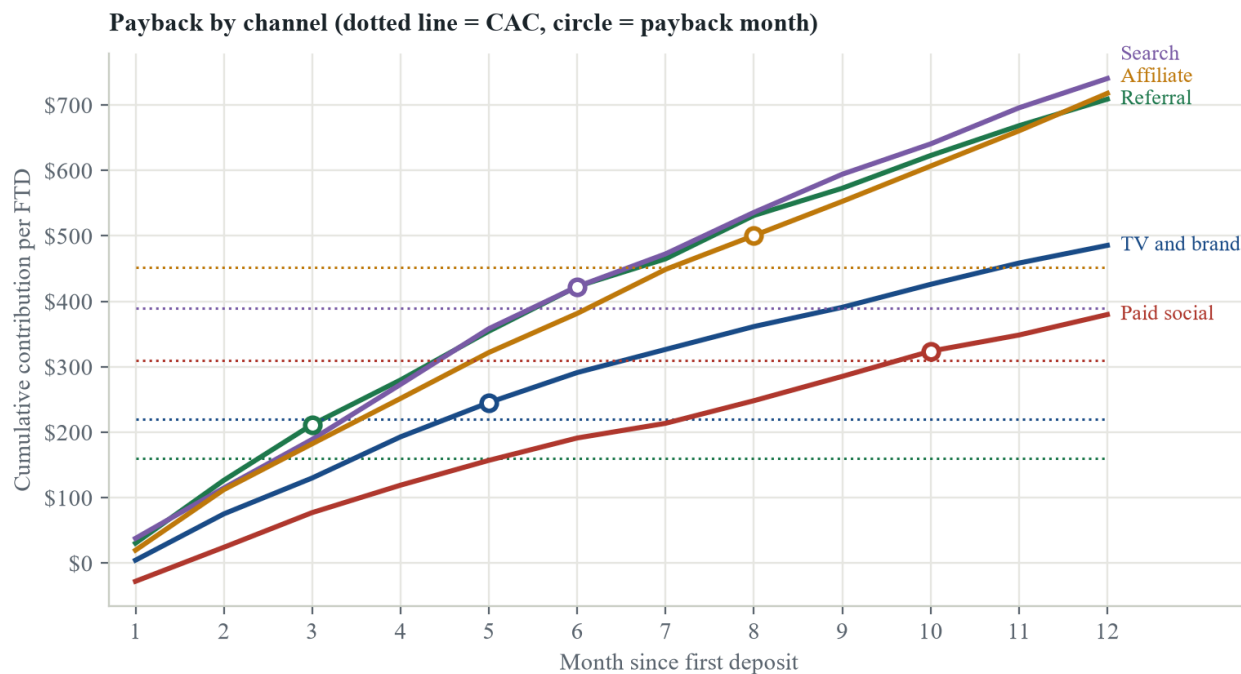
Metric	Squared-error XGBoost	Two-part XGBoost
Top-decile prediction $\div$ actual	0.74	0.85
Spearman, 180-day value	0.572	0.582
Mean absolute error	\$379	\$359
Top-decile lift	10.20×	10.27×
High-value share of top decile	72%	71%

The two-part model closes much of the gap at the top, from 0.74 to 0.85 of the actual top-decile value, and improves ranking and average error, while finding about the same share of true high-value players. Its classifier separates eventually profitable players at 0.869 AUC. The smearing factor of 1.70 is large, which says the log-scale residuals are wide: individual predictions are noisy even when decile averages are right. Both value models predict a lower average than the test cohorts actually produced (\$230 against \$283), so predicted dollars should be recalibrated on recent cohorts before being used as bids.



Calibration. Both retention models beat the base rate on Brier score (0.200 for XGBoost, 0.200 for logistic regression, 0.249 for predicting the training average). XGBoost's probabilities are slightly compressed: its lowest decile predicts 28% and sees 24%, and its highest predicts 88% and sees 91%. Logistic regression tracks the diagonal more closely, which is one more reason to prefer it for retention scoring. On value, the two-part model's deciles sit close to the diagonal while the squared-error model's sit below it.

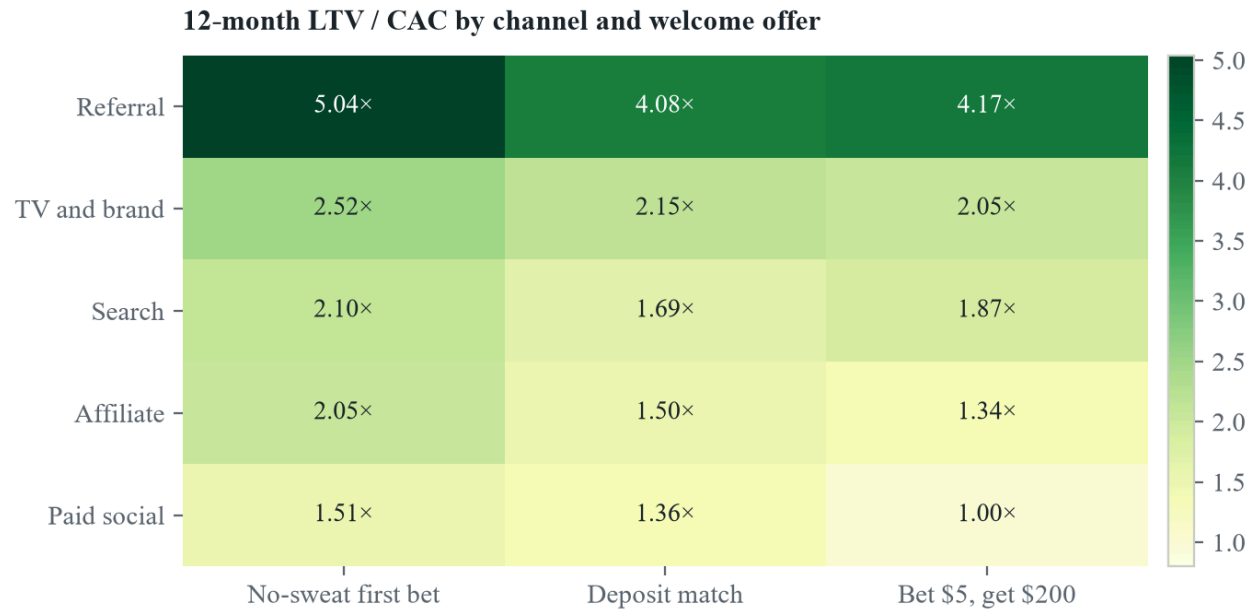
7.2 Channel economics



Channel	FTDs	CAC	12-mo LTV	Median LTV	LTV / CAC	95% interval	Active month 3	Payback
Referral	4,379	\$160	\$709	-\$27	4.42×	3.70 to 5.26	62%	month 3
TV and brand	6,551	\$220	\$485	-\$36	2.21×	1.91 to 2.56	58%	month 5
Search	7,287	\$390	\$740	-\$33	1.90×	1.69 to 2.16	59%	month 6
Affiliate	8,101	\$452	\$718	-\$51	1.59×	1.41 to 1.78	52%	month 8
Paid social	10,139	\$310	\$380	-\$53	1.23×	1.05 to 1.42	50%	month 10

Every channel pays back inside a year, from month 3 for referral to month 10 for paid social, and every interval stays above 1.0×. The median FTD is net negative in every channel. Channels differ in how many high-value players they bring, not in how the typical player behaves.

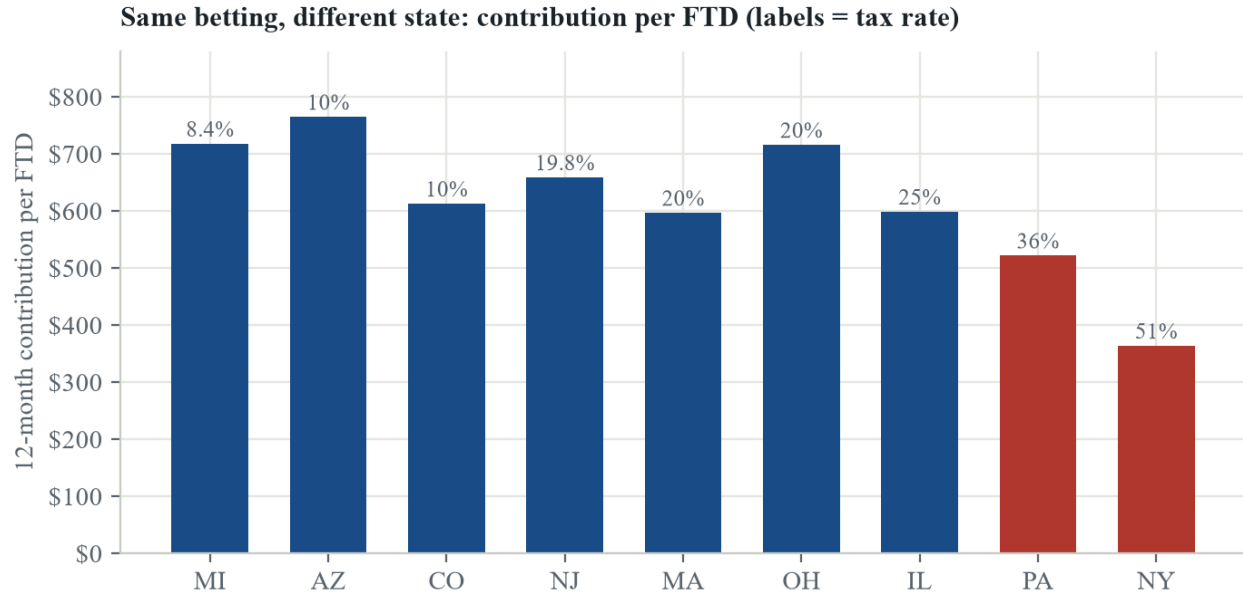
7.3 Channel and offer



Offer	FTDs	Offer cost per FTD	Active month 3	12-mo LTV	LTV / CAC
No-sweat first bet	10,937	\$90	58%	\$698	2.16×
Deposit match	7,314	\$65	55%	\$562	1.74×
Bet \$5, get \$200	18,206	\$144	53%	\$527	1.63×

The channel matters most, but within a channel the offer still moves the return: paid social goes from 1.00× with bet \$5 get \$200 to 1.51× with the no-sweat offer. The direction of this result is an assumption (I assumed the richest offer draws more bonus hunters). The size of the gap, net of offer cost, is the output.

7.4 State tax and bid caps

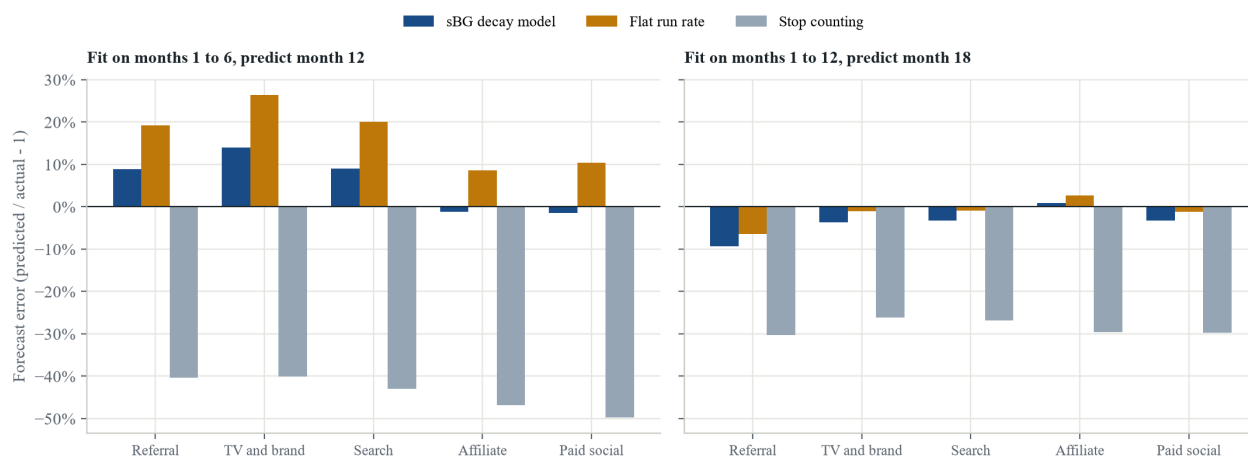


A New York FTD contributes \$364 in year one at 51% tax, against \$718 in Michigan at 8.4%. That gap is arithmetic, but it implies bid caps should be set by state. The table gives the highest CAC that still pays back inside 12 months for each channel and state. **Bold** marks cells where the channel's current CAC is above the cap. Cells hold roughly 300 to 1,600 FTDs, so small differences are noise.

Channel	NY 51%	PA 36%	IL 25%	MA 20%	OH 20%	NJ 19.8%	AZ 10%	CO 10%	MI 8.4%
Referral (CAC \$160)	\$501	\$716	\$668	\$549	\$679	\$694	\$1,304	\$660	\$987
TV and brand (CAC \$220)	\$261	\$379	\$622	\$328	\$755	\$638	\$549	\$440	\$595
Search (CAC \$390)	\$498	\$640	\$706	\$914	\$786	\$989	\$984	\$592	\$861
Affiliate (CAC \$452)	\$426	\$655	\$773	\$771	\$889	\$528	\$1,137	\$860	\$898
Paid social (CAC \$310)	\$227	\$354	\$331	\$407	\$512	\$523	\$258	\$506	\$454

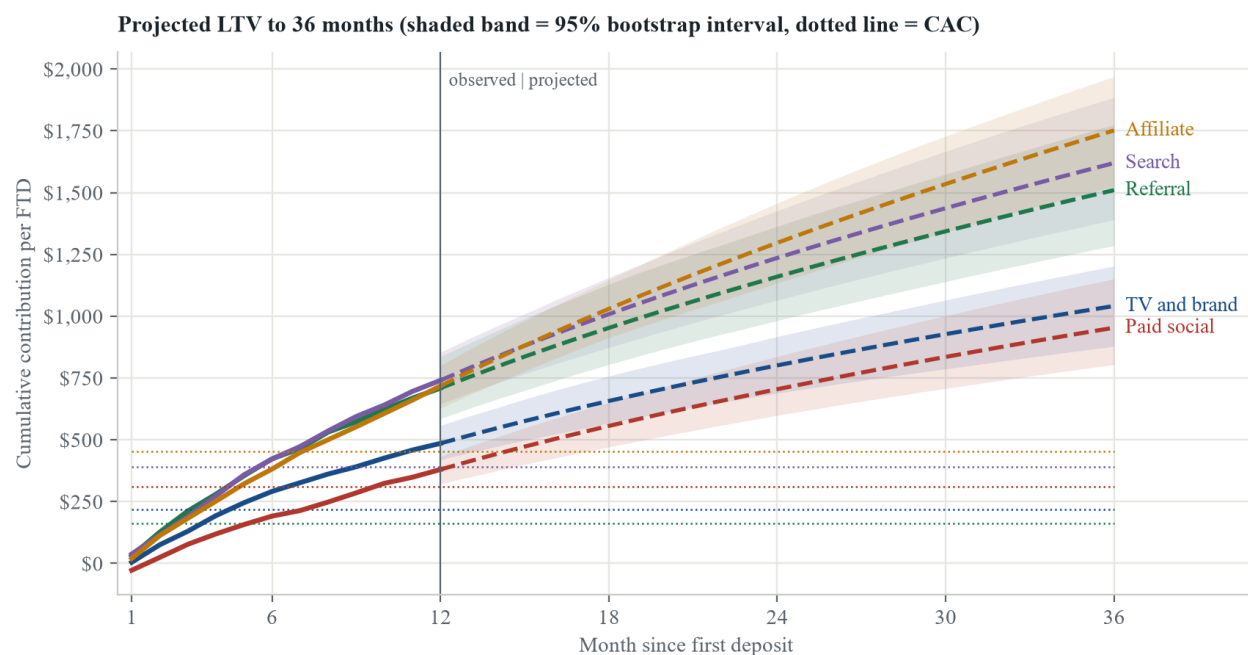
7.5 Lifetime value beyond 12 months

Twelve months understates what a player is worth, because a meaningful share are still betting at month 12. Before projecting further, I tested the projection method on months it never saw.



Backtest	sBG decay model	Flat run rate	Stop counting	sBG, mean channel error
Fit months 1 to 6, predict month 12 (36,457 FTDs)	+5.1%	+16.0%	-44.4%	6.9%
Fit months 1 to 12, predict month 18 (20,012 FTDs)	-3.2%	-0.9%	-28.5%	4.1%

Early on, the decay model is clearly best: from six months of data it lands +5.1% from the actual 12-month value, where a flat run rate overshoots by +16.0% and ignoring the future misses by -44.4%. By month 12, value per player has flattened enough that a flat run rate does about as well over the next six months (-0.9% against -3.2%). But a run rate never decays, so it cannot be stretched to 36 months. The decay model can, and its backtest errors bound how far to trust it.

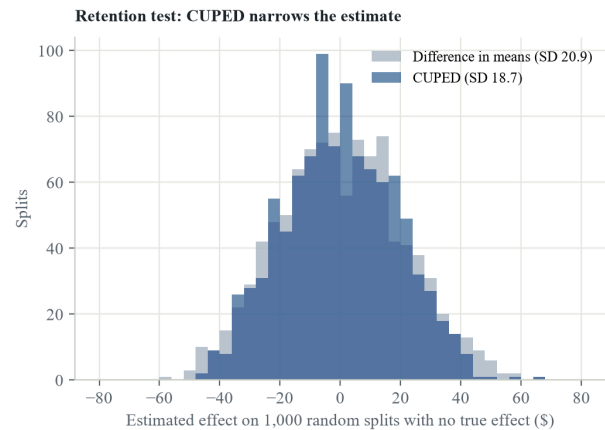
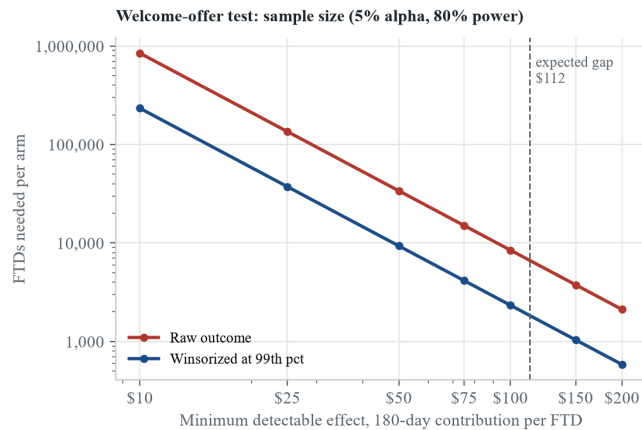


Channel	CAC	12-mo LTV	24-mo LTV	36-mo LTV	36-mo LTV / CAC	Still betting, month 36
Referral	\$160	\$709	\$1,160 (\$982 to \$1,363)	\$1,510 (\$1,286 to \$1,775)	9.42× (8.03 to 11.07)	18%
TV and brand	\$220	\$485	\$801 (\$689 to \$917)	\$1,041 (\$878 to \$1,202)	4.74× (4.00 to 5.47)	16%
Search	\$390	\$740	\$1,236 (\$1,065 to \$1,426)	\$1,620 (\$1,390 to \$1,885)	4.16× (3.56 to 4.84)	17%
Affiliate	\$452	\$718	\$1,297 (\$1,134 to \$1,457)	\$1,752 (\$1,526 to \$1,969)	3.88× (3.38 to 4.36)	15%
Paid social	\$310	\$380	\$704 (\$599 to \$836)	\$953 (\$805 to \$1,150)	3.08× (2.59 to 3.70)	13%

Projected to 36 months, returns range from 9.4× for referral to 3.1× for paid social. The channel ranking is the same as at 12 months. The practical use is setting CAC targets: a team that requires payback inside 12 months is leaving value on the table for channels whose players keep betting, and the intervals show how much of that value is reliable. The projection holds value per active player flat and assumes churned players never return. The later backtest came in 3% low, which suggests the long-run numbers lean conservative, but beyond 18 months they are untested.

8. Test design for the recommended changes

Two recommendations need experiments before anyone acts on them: switching the welcome offer, and targeting retention spend with the model. This section sizes both tests. The two cases differ in one important way: what the analysis is allowed to adjust for.



8.1 Welcome-offer test

New depositors would be randomized at signup between the no-sweat offer and bet \$5 get \$200, with 180-day contribution as the outcome. The expected difference, from the simulation, is \$112 per FTD. The outcome's standard deviation is \$2,321, seven times its mean, so a plain test is expensive.

Minimum detectable effect	Raw outcome	Winsorized at 99th pct	Winsorized and adjusted
\$25	135,304	37,315	37,104
\$50	33,826	9,329	9,276
\$100	8,457	2,333	2,319
\$150	3,759	1,037	1,031

- **Winsorizing** the outcome at the 99th percentile (\$8,311) cuts the required sample by about 72%: to detect the expected \$112 gap, 1,843 FTDs per arm instead of 6,683. The cost is a slightly different estimand, the effect on capped value, which should be stated up front.
- **Covariate adjustment barely helps here.** Only information known before randomization is allowed, which for a new signup means channel, state, and signup month. Together they explain 0.6% of the variance.
- **Early betting cannot be used as a covariate**, even though it would explain 34% of the variance. The offer changes how people bet in their first two weeks, so adjusting for that behavior would absorb part of the very effect being measured.
- **Duration.** 3,686 FTDs in total is about 1.1 months of signups across all channels, or 4.0 months of paid social alone, plus 180 days to observe the outcome. The day-14 value model could provide an early read, but only as a leading indicator, not as the decision metric.

8.2 Retention-spend test

Existing players who bet in month 3 would be randomized to receive a retention offer or not at the start of month 4, with contribution in months 4 to 9 as the outcome (22,050 such players in the data). Here the first three months happened before randomization, so they are valid covariates. CUPED adjusts each player's outcome by their pre-period contribution; regression adjustment uses several pre-period measures.

Adjustment	Variance reduction, formula	Variance reduction, 1,000 random splits
CUPED, pre-period contribution	20%	20%
Regression, four pre-period measures	26%	22%

Minimum detectable effect	Unadjusted	CUPED	Regression
\$25	59,168	47,516	43,876
\$50	14,792	11,879	10,969
\$100	3,698	2,970	2,743
\$150	1,644	1,320	1,219

The formula and the simulation agree: adjusting for the pre-period cuts variance by about a fifth, which cuts the required sample by the same share. The right panel of the figure shows it directly. Across 1,000 random splits with no true effect, the CUPED estimate's standard deviation is \$19 against \$21 for a plain difference in means. Adding a covariate is free once the data exists, so there is no reason to run this test without it.

9. Conclusion

9.1 Summary

An FTD in this simulation returns $1.81\times$ its acquisition cost in year one, but that average hides extreme concentration: 46% of contribution comes from 1% of players and most players lose money after promos. The biggest levers are which channel a player comes from, which offer they receive, and which state they bet in. Two weeks of behavior is enough to rank players usefully, and a simple model on those two weeks ranks them better than early handle alone, especially with a two-part model. Projected with a retention-decay model that backtests within a few percent, 36-month returns run from $3.1\times$ to $9.4\times$ CAC. Section 10 tests a pretrained tabular model, TabPFN, on the same problem.

9.2 Recommendations

1. Test the no-sweat offer against bet \$5 get \$200 before changing any channel's budget: about 1,843 FTDs per arm with a winsorized outcome (section 8.1).
2. Set acquisition bid caps by state (7.4) rather than one national CAC target, and base them on projected rather than 12-month value where the backtest supports it (7.5).
3. Score every FTD at day 14 with the two-part value model, recalibrated on recent cohorts, for retention spend across the whole base. Use a handle threshold to route likely high-value players to VIP.
4. Run the retention-spend test with CUPED on pre-period contribution; it cuts the required sample by about a fifth (section 8.2).
5. Report channel LTV with an interval. With value this concentrated, a few players can move a channel's average.

6. Use logistic regression for retention scoring: it matches XGBoost and is easier to explain.

9.3 Limitations

- Player-level data is simulated. Channel, offer, and player-type effects come from my assumptions, so conclusions about them demonstrate the method rather than describe real DraftKings economics.
- Only seasonality, hold, hold volatility, and New York's tax rate are calibrated to real data, and only from New York, the highest-tax state.
- Tax is a flat rate on GGR net of promos. Real states tier it, tax per wager, or limit promo deductions, and the non-New York rates are approximate.
- No casino or daily fantasy cross-sell, and churned players never return.
- LTV projections beyond 18 months are extrapolations. The backtests cover 6 to 12 and 12 to 18 months only, and the projection holds value per active player flat.
- Sample sizes assume the simulated variance. Real outcome variance, and so the real required sample, should be measured on recent cohorts before a test launches.

10. TabPFN: process and comparison

10.1 What TabPFN is

TabPFN (Hollmann et al., *Nature*, 2025) is a transformer pretrained by Prior Labs on millions of synthetic datasets drawn from a prior over how tabular data is generated: random causal structures, noise, missing values, and mixed types. It does not fit parameters to a new dataset. The labeled training rows go into the model as context, and it predicts the unlabeled rows in a single forward pass, in effect performing approximate Bayesian inference learned during pretraining. There is no hyperparameter tuning.

The practical claim is strong accuracy on small and medium tables, where there is not enough data to train and tune a model well. That is a common situation in customer economics: a new state launch, a new promotion, or a new product, with a few thousand players and a decision due before more data arrives.

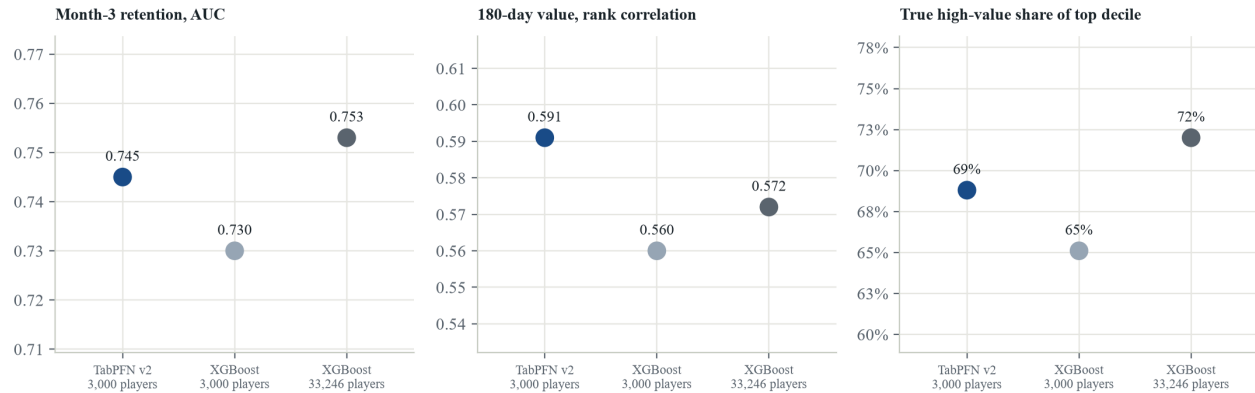
10.2 Setup

- **Model version.** The open TabPFN v2 weights from Hugging Face (`Prior-Labs/TabPFN-v2-clf` and `-reg`), loaded through the `tabpfn` package. Newer versions (3.5) require an account and license acceptance for local use, so I used v2, which is ungated.
- **Same problem as section 6.** Same features, same targets, same out-of-time test set of 6,754 players, so results are directly comparable.

- **Context size.** 3,000 players sampled at random from the training cohorts. TabPFN v2 was pretrained on datasets up to about 10,000 rows, and inference cost grows with context size, so a smaller context kept the run feasible on a laptop CPU.
- **Encoding.** Categorical columns were integer-coded and flagged as categorical so TabPFN applies its own categorical handling. Numeric features were passed raw: TabPFN does its own preprocessing internally, including transforms suited to skewed data.
- **Ensembling.** 4 ensemble members, each with a different feature ordering and preprocessing, averaged.
- **CPU.** TabPFN refuses CPU runs above 1,000 rows by default because they are slow, so I set `ignore_pretraining_limits=True` and predicted in batches of 1,000. Scoring the 6,754 test players took between about 7 and 25 minutes per model across my runs, depending on what else the laptop was doing. The predictions are cached so the comparison can be rebuilt without rerunning TabPFN.

To separate the effect of the model from the effect of less data, I compared three setups on the same test players: TabPFN on the 3,000-player sample, XGBoost on the same 3,000, and XGBoost on the full training set.

10.3 Results



Model	Retention AUC	Value Spearman	Value MAE	Top-decile lift	High-value share, top decile
TabPFN v2, 3,000 players	0.745	0.591	\$368	10.25×	69%
XGBoost, 3,000 players	0.730	0.560	\$388	9.94×	65%
XGBoost, 33,246 players	0.753	0.572	\$379	10.20×	72%

10.4 Interpretation

- **At equal data**, TabPFN beats XGBoost on retention by 0.015 AUC, and ranks 180-day value at 0.591 Spearman against 0.560.
- **Against 11× the data**, XGBoost on 33,246 players: TabPFN is within 0.008 on retention AUC, and scores 0.591 against 0.572 on value ranking. On value ranking, the model with a tenth of the data is the best of the three.
- **Why value ranking goes this way**. XGBoost with squared-error loss spends much of its capacity fitting the few very large players, so its ranking of everyone else gains little from more data (0.560 on 3,000 players, 0.572 on all of them). TabPFN predicts from a learned prior over tables rather than minimizing squared error on this one, which seems to make it less sensitive to the tail. Modeling log value with XGBoost would be the fair next comparison.
- **Against the two-part model**. Fixing XGBoost's loss (section 7.1) lifts its value ranking to 0.582 on the full data, which narrows but does not close the gap to TabPFN's 0.591. Part of TabPFN's edge was the loss function; part of it was not.
- **Finding the top players**. Top-decile lift is nearly identical across all three (10.25×, 9.94×, 10.20×). The share of true high-value players in each model's top decile is 69%, 65%, and 72%.
- **Cost**. XGBoost trains and scores in seconds. TabPFN needed minutes per model on CPU, because every prediction attends over the whole context. At production scale it wants a GPU or Prior Labs' hosted API.
- **Where I'd use it**. First, small-sample questions where there is no time or data to tune a model: early reads on a new state, offer, or product. Second, as a benchmark. Its value-ranking result says the full-data XGBoost model is leaving accuracy on the table, and the next step would be to fix that model's loss rather than assume more data solves it.

TabPFN v2 weights are released under the Prior Labs License, which is Apache 2.0 with an attribution requirement. Built with TabPFN.

11. Reproducibility

```
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt
.venv/Scripts/python run.py           # simulate, SQL, models, LTV projection,
test design, report (under a minute)
.venv/Scripts/python model_tabpfn.py  # TabPFN comparison (15 to 40 minutes on
CPU the first time, then cached)
.venv/Scripts/python build_report.py  # rebuild this report with the TabPFN
results
```

The pipeline is deterministic. One bug worth recording: results first changed between runs because DuckDB does not guarantee row order out of a parallel join, and row order decides which rows XGBoost holds out for early stopping. Sorting the feature table fixed it.